TD4 : React

Corrigé des exercices

Exercice 1 : SuperCounter

Composant SuperCounter (compteur unique)

```
import React, { useState } from 'react';

function SuperCounter() {
  const [count, setCount] = useState(0);
  const [step, setStep] = useState(1);

  const increment = () => {
    if (count + step <= 20) setCount(count + step);
  };

  const decrement = () => {
    if (count - step >= 0) setCount(count - step);
  };

  const reset = () => setCount(0);

  const getColor = () => {
    if (count > 0) return 'green';
    if (count < 0) return 'red';
    return 'black';
  };

  return (
    <div>
      <h1 style={{ color: getColor() }}>{count}</h1>
      <div>
        <button onClick={decrement} disabled={count - step < 0}>-</
           button>
        <button onClick={increment} disabled={count + step >
           20}>+</button>
        <button onClick={reset}>Reset</button>
      </div>
      <div>
        <label>Step : </label>
        <input
```

```
          type="number"
          value={step}
          onChange={(e) => setStep(Number(e.target.value))}
          min="0"
          max="20"
        />
      </div>
    </div>
  );
}

export default SuperCounter;
```

Deux compteurs indépendants + total

```
import React, { useState } from 'react';
import SuperCounter from './SuperCounter';

function App() {
  const [total1, setTotal1] = useState(0);
  const [total2, setTotal2] = useState(0);

  return (
    <div>
      <SuperCounter onCountChange={setTotal1} />
      <SuperCounter onCountChange={setTotal2} />
      <h2>Total des deux compteurs : {total1 + total2}</h2>
    </div>
  );
}

export default App;
```

Modification de SuperCounter pour remonter la valeur

```
import React, { useState, useEffect } from 'react';

function SuperCounter({ onCountChange }) {
  const [count, setCount] = useState(0);
  const [step, setStep] = useState(1);

  // ...

  useEffect(() => {
    if (onCountChange) onCountChange(count);
  }, [count, onCountChange]);

  // ... (reste identique)
}
```

Exercice 2 : Formulaire avec validation

```
import React, { useState } from 'react';

function Formulaire() {
  const [nom, setNom] = useState('');
  const [prenom, setPrenom] = useState('');
  const [age, setAge] = useState('');
  const [valide, setValide] = useState(false);

  const handleSubmit = (e) => {
    e.preventDefault();
    if (nom && prenom && age) {
      setValide(true);
    } else {
      // alert('Veuillez remplir tous les champs');
      setValide(false);
    }
  };

  return (
    <div>
      <form onSubmit={handleSubmit}>
        <div>
          <label>Nom : </label>
          <input value={nom} onChange={(e) => setNom(e.target.value
              )} />
        </div>
        <div>
          <label>Prenom : </label>
          <input value={prenom} onChange={(e) => setPrenom(e.target
              .value)} />
        </div>
        <div>
          <label>Age : </label>
          <input type="number" value={age} onChange={(e) => setAge(
              e.target.value)} />
        </div>
        <button type="submit">Valider</button>
      </form>

      {valide && (
        <p>Bienvenue {prenom} {nom}, vous avez {age} ans.</p>
      )}
    </div>
  );
}

export default Formulaire;
```

Exercice 3 : Liste filtrée en temps réel

```
import React, { useState } from 'react';

const nomsInitiaux = ['Ali', 'Sami', 'Ahmed', 'Yasmine', 'Salma', '
   Karim'];

function ListeFiltree() {
  const [recherche, setRecherche] = useState('');

  const nomsFiltres = nomsInitiaux.filter(nom =>
    nom.toLowerCase().includes(recherche.toLowerCase())
  );

  return (
    <div>
      <input
        type="text"
        placeholder="Rechercher un nom..."
        value={recherche}
        onChange={(e) => setRecherche(e.target.value)}
      />

      <ul>
        {nomsFiltres.length > 0 ? (
          nomsFiltres.map((nom, index) => <li key={index}>{nom}</li
             >)
        ) : (
          <li>Aucun resultat</li>
        )}
      </ul>
    </div>
  );
}

export default ListeFiltree;
```

Récapitulatif des concepts utilisés

Exercice	Concepts React
1	useState, événements (onClick, onChange), props, remontée de state, styles dynamiques, useEffect
2	useState, formulaire contrôlé, validation, affichage conditionnel
3	useState, filtrage en temps réel, map(), filter(), affichage conditionnel